Python Engineering

Class 07 - NumPy Arrays | EX15b - Shapes & Operations Practice

No week or class change. We remain in **week_02_data_structures -> class_07_numpy_arrays**. This extra exercise is designed to build fluency before moving on.

Goal

Deliberately practice creating 1D and 2D arrays, inspecting dimensions, indexing them, checking operation compatibility, and reasoning about matching shapes.

BLOCK A - Create arrays and inspect geometry

```
import numpy as np

vector_a = np.array([1.0, 2.0, 3.0])
vector_b = np.array([4.0, 5.0, 6.0])

matrix_a = np.array([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
])

matrix_b = np.array([
    [10.0, 20.0, 30.0],
    [40.0, 50.0, 60.0]
])
```

For each object print `shape`, `ndim`, `size`, and `dtype`. Predict them before running.

BLOCK B - Indexing and slicing

```
matrix_a[0, 0]
matrix_a[1, 2]
matrix_a[0, :]
matrix_a[:, 1]
matrix_a[-1, :]
matrix_a[:, 1:]
```

For every expression, predict both the returned values and the result shape.

BLOCK C - Element-wise operations

```
vector_a + vector_b
vector_a - vector_b
vector_a * vector_b
vector_a / vector_b
vector_a**2
matrix_a + matrix_b
matrix_a * matrix_b
matrix_a * 10
```

`*` is element-wise multiplication here, not matrix multiplication.

BLOCK D - Broadcasting

```
offset = np.array([100.0, 200.0, 300.0])
result = matrix_a + offset
```

NumPy can apply a shape `(3,)` vector to every row of a `(2,3)` matrix. This is **broadcasting**.

```
bad_offset = np.array([10.0, 20.0])
matrix_a + bad_offset
```

Observe the error and explain why shapes `(2,3)` and `(2,)` are incompatible here.

BLOCK E - Reshape and transpose

```
row = vector_a.reshape(1, 3)
column = vector_a.reshape(3, 1)
matrix_t = matrix_a.T
```

Object	Expected shape
vector_a	(3,)
row	(1,3)
column	(3,1)
matrix_a	(2,3)
matrix_a.T	(3,2)

BLOCK F - Concatenation

```
new_row = np.array([[7.0, 8.0, 9.0]])
matrix_rows = np.vstack((matrix_a, new_row))

new_column = np.array([[10.0], [20.0]])
matrix_columns = np.hstack((matrix_a, new_column))
```

For `vstack`, column counts must match. For `hstack`, row counts must match.

BLOCK G - Maxima along axes

```
np.max(matrix_b)
np.max(matrix_b, axis=0)
np.max(matrix_b, axis=1)

np.argmax(matrix_b)
np.argmax(matrix_b, axis=0)
np.argmax(matrix_b, axis=1)
```

Interpret which dimension is being reduced instead of memorizing results.

BONUS - Dot product and matrix-vector product

```
dot_product = vector_a @ vector_b
matrix_vector = matrix_a @ vector_b
```

@ represents linear/matrix multiplication, not element-wise multiplication. We will study linear algebra more deeply later.

FINAL CHECK

```
Explain:
- Why (3,) is not the same as (1,3)
- Why (2,3) + (3,) works
- Why (2,3) + (2,) fails
- Why vstack needs matching columns
- Why hstack needs matching rows
- Difference between * and @
- Difference between axis=0 and axis=1
```

Completion criterion

The goal is not memorization. EX15b is complete when you can look at two shapes and reason whether an operation makes sense before running it.